

ОСНОВНОЙ ЭКРАН РОБОТА (КИОСК): ДВИГАТЕЛИ, ГРУЗОВОЙ ОТСЕК, АКБ

Документ: docs/MAIN_SCREEN.md **Файлы:** gui/main.html, gui/main.css, gui/main.js, gui/backend.py, gui/serve.py **Версия:** 2.0 (визуал + бэкенд) · **Дата:** 2026-10 **Связанные документы:** [RUN_AND_ACCESS.md](#) (как попадать на экраны) · [GUI.md](#) · [SERVICE_CONSOLE.md](#) · [SERIAL_PROTOCOL.md](#) · [SOFTWARE_ARCHITECTURE.md](#)

Это экран, который видят на самом роботе: открывается автоматически при запуске сервера на борту и не требует прокрутки, клавиатуры и мыши — только касания.

1. Запуск на работе

```
python3 gui/serve.py # == python3 gui/backend.py
python3 gui/serve.py --kiosk # + браузер без рамок на весь дисплей
python3 gui/serve.py --source serial --ports /dev/ttyUSB0,/dev/ttyUSB1,/dev/ttyUSB2,/dev/ttyUSB3
python3 gui/serve.py --source ros # данные из ROS 2 (rclpy)
python3 gui/serve.py --pin 2580 --reset-pin
```

Что делает --source:

Значение	Источник	Когда использовать
sim (по умолчанию)	демонстрационная физика внутри бэкенда	стенд, показ, отладка экрана без робота
serial	4 UART, кадры телеметрии 16 Б (CRC16/Modbus), нужен pyserial	реальные модули напрямую с бортового ПК
ros / auto	ROS 2: /screen/state, /battery/voltage, /battery/current, /screen/motors	штатная работа через ROS 2; при отсутствии rclpy автоматически падает в sim

Адреса печатаются в консоли: http://127.0.0.1:8080/main.html (сам робот) и http://<ip-робота>:8080/main.html (планшет рядом). Кэш отключён (Cache-Control: no-store) — после обновления файлов достаточно перезагрузить страницу.

Автозапуск (systemd, графическая сессия):

```
[Unit]
Description=RUS SLAM main screen

[Service]
ExecStart=/usr/bin/python3 /opt/rus_slam/gui/serve.py --kiosk
Restart=always

[Install]
WantedBy=graphical.target
```

2. Компоновка экрана



Блоки:

Блок	Что показывает	Крупные элементы
Двигатели	4 модуля (FL/FR/RL/RR): угол руля на циферблате, об/мин, температура, полоса нагрузки, чип состояния (готов / движение / хоминг / перегрев)	цифры ≥ 21 px, циферблат ≤ 130 px
АКБ	Заряд в процентах кольцом (центр — число), напряжение, ток, запас хода, состояние, подсказка о порогах 35,5 / 33,5 В	% — до 64 px
Грузовой отсек	Состояние замка (ЗАКРЫТО / ОТКРЫТО / БЛОКИРОВКА), маска PIN, сенсорная клавиатура 0-9/⌫/СБРОС, кнопка «Открыть отсек», масса груза	клавиши ≥ 44 px, PIN — до 40 px

Шапка: название, режим, индикатор связи с модулями, часы. Подвал: маршрут, оценка мощности тяги, подсказка про инженерный пульт.

3. Что уже работает (визуальная часть)

- сенсорная клавиатура: ввод PIN точками, «СБРОС», подтверждение «Открыть отсек»;
- проверка PIN 2580 (демо), сообщение об ошибке, **5 неудач → блокировка 30 с** с обратным отсчётом и запретом кнопки;
- перевод состояния замка в ОТКРЫТО и отметку «груз доступен»;
- живые показания двигателей (угол, об/мин, температура, чип состояния);
- кольцо заряда с цветовой индикацией: норма → низкий заряд → аварийный;
- часы, индикатор связи, счётчик «в движении: N из 4».

Числа сейчас демонстрационные (плавно меняются), чтобы согласовать вид.

4. Бэкенд (gui/backend.py)

Один процесс отдаёт статику обоих экранов, JSON API и работает с замком. Без внешних зависимостей (кроме pyserial для --source serial и ROS 2 для --source ros).

4.1. HTTP API

Метод и путь	Назначение	Ответ
GET /api/state	снимок состояния (экран опрашивает 300 мс)	{ok, data{...}}
POST /api/lock/open	открыть отсек по {"pin":"..."}	{ok:true, lock{}} или 403 {ok:false, reason, attemptsLeft}
POST /api/lock/close	закрыть отсек	{ok:true, lock{ }, hardware}
POST /api/lock/pin	сменить PIN: {"current":"...", "new":"..."}	{ok:true} или 403 {ok:false, detail}
GET /api/audit?limit=N	журнал доступа и целостность цепочки	{ok, audit[], integrity{}}
GET /api/health	живость, источник данных, время работы	{ok, source, uptimeSec}
GET / · GET /console	302: на основной экран · на инженерный пульт	—

Состав data в /api/state:

```
motors[] : id, title, angle (град), rpm, temp (°C), homed, online, fault
battery  : soc (%), volts, amps, watts, remainingWh, rangeKm, state, level
cargo    : kg, closed
lock     : open, blocked, remainingMs, attemptsLeft, changedAt
прочее   : mode, route, powerKw, linkOk, source, uptimeSec, ts
```

Примеры запросов:

```
curl -s http://127.0.0.1:8080/api/state | python3 -m json.tool | head -20
curl -s -X POST http://127.0.0.1:8080/api/lock/open \
  -H 'Content-Type: application/json' -d '{"pin":"2580"}'
```

4.2. Данные экрана: откуда берутся

Данные	sim	serial (UART)	ros
Угол, об/мин, состояние модуля	модель	поля кадра 16 Б	/screen/motors
Напряжение и ток	модель	усреднение по 4 модулям	/battery/*
Заряд в процентах	по расходу энергии	по OCV-кривой 12S3P	по OCV-кривой
Запас хода	714,2 Вт·ч ÷ расход	то же	то же
Уровень заряда	пороги 35,5 / 33,5 В	то же	то же
Груз, маршрут, режим	константы демо	состояние	/screen/state

Пороговые значения и OCV-кривая совпадают с gui/console-core.js (RS.pack) — экран и сервисный пульт показывают одинаковые цифры.

4.3. Замок грузового отсека

- PIN: SHA-256(salt + PIN), файл gui/state/lock.json (соль, хеш, счётчик попыток, время блокировки, журнал аудита с хеш-цепочкой);
- 5 неудачных попыток → блокировка 30 с (настраивается --max-attempts, --lock-sec), проверка блокировки — на сервере, экран только показывает;
- реальное железо подключается переменными окружения:

```
# открыть отсек: включить реле/соленоид на 2 с
export RUS_SLAM_LOCK_CMD='gpio write 7 1 && sleep 2 && gpio write 7 0'
# закрыть отсек
export RUS_SLAM_LOCK_CMD_CLOSE='gpio write 7 0'
python3 gui/backend.py
```

Если переменные не заданы — замок программный (`hardware:"software"`), а экран и API работают полностью, что удобно для стенда и демонстрации.

4.4. Что ещё предстоит подключить

- фактические датчики массы груза и температуры модулей (сейчас в кадрах телеметрии их нет — поля заполняются моделью);
- публикацию `/screen/*` из `mission_manager` и `module_bridge`, чтобы `--source ros` работал «из коробки» на работе;
- аварийную индикацию: красная подсветка блока и звук при E-STOP, потере связи и напряжении $< 33,5$ В;
- передачу событий экрана (открытия отсека, ошибки) в статистику смены `RS.StatsStore` для отчёта.

5. Отношение к инженерному пульта

Страница	Назначение	Открывается
<code>main.html</code>	основной экран на работе (киоск): моторы, отсек, АКБ	корень <code>/</code> , автозапуск
<code>index.html</code>	инженерный пульт: карта SLAM, миссия, сервисное окно, статистика	<code>/index.html</code> , <code>/console</code>

Обе страницы независимы: сбой одной не влияет на другую, общие файлы — только `main.css/main.js` и `console-*` для пульта.

6. Проверка

```
python3 gui/backend.py --port 8081 # вручную в браузере
python3 gui/tests/backend.test.py # 36 проверок бэкенда и API
NODE_PATH=/tmp/harness/node_modules node gui/tests/main.screen.test.js # 27 проверок экрана
```

- `backend.test.py` — CRC16 и кадры протокола, разбор потока UART с мусором, модель АКБ (пороги, OCV), PIN-хранилище (попытки, блокировка, смена PIN, целостность аудита, персистентность), живой HTTP API (статика, `/api/state`, открытие/закрытие замка, журнал, смена PIN);
- `main.screen.test.js` — разметка, живые показания, кольцо АКБ в процентах, ввод PIN, открытие/закрытие, блокировка после пяти неудач, режим «демо» при отсутствии сервера, ссылка на инженерный пульт;
- `main.api.test.js` — сквозной прогон «экран ↔ бэкенд»: живой `backend.py`, данные `/api/state` на экране, открытие отсека по PIN через API, аудит, закрытие и блокировка после пяти неудач (проверка считает сервер).

Всего по проекту GUI: **207 проверок** — 32 (ядро сервисного пульта) + 93 (пульт в jsdom) + 36 (бэкенд и API) + 27 (экран) + 19 (экран + API).

Как попадать на экраны (киоск, планшет, переходы) — `docs/RUN_AND_ACCESS.md`.

7. Лицензия и авторы

MIT, команда RUS SLAM, НТЦ АО «АВТОВАЗ», конкурс «Первые в профессии», задание № 2.